AI WEB 开发挑战赛 · 技术文档

Margin
研发辅助 Agent 工作台

一个工作台 + 多个可插拔 AI 研发助手
代码审查 · 架构设计

赛题方向 7 · AI 代码开发辅助

参赛组别：【研究生组 / A 组 / B 组 / C 组】

学校：【学校全称】

团队成员：【姓名 1（队长）、姓名 2、姓名 3】

指导教师：【姓名 1、姓名 2】

交付版本：见仓库 Git 提交历史 · 仓库地址：github.com/15667h/Code-review-agent

目录

1 **项目概述** 1.1 一句话说明 · 1.2 要解决的问题 · 1.3 产品形态 · 1.4 与同类工具的差异

2 **需求分析** 2.1 目标用户与场景 · 2.2 功能需求 · 2.3 非功能需求 · 2.4 赛题评分维度映射

3 **系统架构设计** 3.1 总体架构 · 3.2 Agent = Model + Harness · 3.3 前端工作台 · 3.4 事件流
3.5 关键架构决策 · 3.6 技术选型理由

4 **AI 技术方案** 4.1 模型集成 · 4.2 Agent 循环 · 4.3 Prompt 工程 · 4.4 工具接口设计
4.5 上下文预算与压缩 · 4.6 缓存感知的上下文结构 · 4.7 安全与可控性
4.8 规则与模型的交叉验证 · 4.9 结构化产出

5 **核心功能实现** 5.1 代码审查助手 · 5.2 架构设计助手 · 5.3 多助手隔离
5.4 账号与会话安全 · 5.5 前端设计系统

6 **测试与部署** 6.1 测试体系 · 6.2 评估体系 · 6.3 架构助手难度实测 · 6.4 部署与运行

7 **创新点说明** 7.1 技术创新点 · 7.2 应用创新点

8 **总结与展望** 8.1 已交付能力 · 8.2 已知边界与不足 · 8.3 后续计划

1 项目概述

1.1 一句话说明

Margin 是一个面向研发流程的 AI 工作台：它把「读代码、找问题、讲结构」这类需要长时间专注的工作交给一组可插拔的 AI 助手，并用一套统一的界面承载它们的产出。 当前内置两个助手 —— **代码审查**（产出带证据的发现列表）与 **架构设计**（产出可评审的分层蓝图）。

1.2 要解决的问题

研发过程中有两类高价值但耗时的智力劳动，长期依赖资深工程师的时间：

问题	传统做法的困难	本项目的做法
代码审查 找缺陷、看资源生命周期、查边界条件	人工逐文件阅读，成本高且不稳定；简单静态扫描工具只能覆盖规则化问题，误报多、无法理解语义	让 Agent 带着工具循环读代码（读文件、搜符号、看 diff、跑规则）， 产出的每条发现都必须带**文件与行号证据**，并按严重级归类
架构理解与设计 接手陌生仓库、评审分层与依赖	需要通读大量代码才能画出结构图；口头结论无法沉淀，评审时缺少可讨论的载体	让只读的架构助手产出**分层图 + 组件清单 + 关键决策 + 开放问题**， 每条决策写明「放弃了什么、代价是什么」，形成可评审的方案草稿

1.3 产品形态

界面是一间「工作室」： 左侧是助手轨道与空间（被审仓库）列表，中间是工作台（对话 + 产出）， 下方是输入坞。切换助手时，不只是换一个图标 —— **产出形态、工具集、界面文案、强调色** 整套都跟着变。



图 1-1 工作台骨架：轨道 + 上下文栏 + 工作台 + 输入坞

1.4 与同类工具的差异

维度	常见的 AI 编程工具	Margin 的做法
定位	编辑器内的代码补全 / 对话	独立工作台，承载**完整的一次审查或设计任务**及其产出
产出	自由文本回复，需人工再整理	**结构化产出**：发现 / 组件 / 决策 / 开放问题各有 schema，前端按形态渲染
证据	可能给出不存在的行号	引用核验：结论中引用的位置若不在本次读过的文件里，会被标出待复核
扩展	新能力需要改代码	**助手是一份声明**（AgentSpec），接入新助手不改既有代码
可信度	难以评估效果	自带评估链路：8 个靶场 + 双口径指标 + 人工复核，数字由脚本现算

1.5 交付规模

29,272
行源码（前端 + 后端 + 测试 + 评估）

175
项回归测试（全绿）

2
个内置 AI 助手（可插拔扩展）

1
个外部依赖（httpx）

2 需求分析

2.1 目标用户与场景

用户	场景	需求本质
开发者（个人）	提交前自查；接手陌生模块时快速建立结构认知	需要一份「可以照着改」的问题清单，而不是一堆风格建议
团队评审者	把 AI 产出当作评审的起点，逐条讨论	需要证据可追溯、结论可反驳、边界清楚（哪些没看）
学生 / 学习者	用真实仓库练习代码阅读与架构分析	需要把「为什么这样设计」讲出来，而不只是给答案

2.2 功能需求

编号	功能	说明与验收标准
F1	选择被审仓库	自动列出示例子仓库，也可手动加入任意本地目录；服务端校验路径不得越界
F2	限定审查范围	可勾选仓库内的相对路径子集；越界项被丢弃且不回显原文
F3	发起智能体运行	输入目标（自然语言），Agent 自主决定读哪些文件、调哪些工具，直到收尾
F4	实时查看过程	SSE 流式推送：每一步工具调用、每一次产出登记即时可见
F5	结构化产出	审查产出发现列表（可按严重级筛选）；架构产出分层图 + 组件 + 决策 + 开放问题
F6	历史会话	运行结果归档，可重放；**按助手隔离**，切换助手只看到该助手的历史
F7	可中断	运行中可停止；被中断的那一轮如实标为未完成
F8	账号系统	注册 / 登录 / 退出；密码加盐哈希存储；可选强制登录
F9	接入新助手	写一份声明即可，不改既有代码；前端自动出现入口、文案与配色

2.3 非功能需求

维度	要求与实现方式
安全 — 密钥	API Key 只存 `.env`（不入库）；一切读取都过密钥文件护栏，且护栏在**工具层**（因为读取结果会进模型上下文，只挡浏览器不够）
安全 — 越界	路径逃逸、绝对路径、技能目录逃逸一律拒绝；静态路由白名单 + 后缀白名单 + 拒绝点目录
安全 — 凭据	密码 scrypt 加盐哈希，明文不落盘；会话 HttpOnly + SameSite=Strict；登录失败不区分「用户不存在」与「密码错」以防账号枚举
可靠性	失败必须显式（目录不存在不能伪装成「没有匹配」）；归档原子写入；坏文件留档不静默丢弃
可审计	零 web 框架：HTTP 层用标准库实现，依赖只有 httpx —— 这个服务本身也要可被审计
可扩展	助手 = 声明；产出形态 = 声明；渲染器 = 注册表 + 兜底，未知形态不白屏也不冒充实出
性能	首屏无构建、无第三方 JS；后端单进程可并发处理 SSE；模型侧靠缓存命中降低延迟与成本
可用性	无 API Key 时降级为离线规则模式，界面功能完整；窄屏自适应到 430px

2.4 赛题评分维度映射

下表说明本项目对赛题各评分维度的对应实现，便于评审定位：

评分维度（权重）	赛题考察点	本项目的实现与位置
AI 技术深度 35%	大模型集成质量 8%	统一的模型客户端（重试 / 退避 / 用量统计）；系统提示分层；产出 schema 强约束；缓存命中率实测 ~87%
	Agent / 工作流设计 8%	完整 Agent 循环：规划（任务列表）→ 工具调用 → 观察 → 收敛（finish）；工具集按助手插拔；熔断与迭代上限护栏
	RAG 技术应用 6%	以**按需检索**替代向量检索：工具清单渐进披露（先检索再加载）、代码按符号索引读取 —— 面向代码场景的检索策略
	多模态能力 5%	当前版本聚焦文本代码理解，未集成图像/语音（见 8.2 已知边界）
	AI 安全与可控性 4%	熔断（连续被拒计数、可恢复）、重复调用检测、引用核验、失败显式化、只读助手硬契约
	MCP / 工具集成 4%	工具即契约：JSON Schema 参数、四层错误信息（类型+参数+建议）、工具描述写「何时用/何时不用」
创新性 25%	选题创新 10%	把「代码审查」与「架构设计」放进同一可插拔内核；架构助手产出可评审蓝图而非泛泛描述
	技术方案创新 8%	确定性规则集兼作模型结果的交叉验证器；缓存感知的上下文结构；零依赖后端
	交互模式创新 7%	助手切换时产出形态/文案/配色整体切换；分层蓝图的可读化（连线几何、决策与开放问题并列）
前端工程质量 10%	代码质量 4%	模块化原生 JS（8 个模块）、统一 UI 原语、无第三方运行时依赖
	组件化与架构 3%	渲染器按产出形态注册；视图注册表统一生命周期；助手轨道/上下文栏/工作台解耦
	性能优化 3%	零构建直出、无阻塞脚本、SSE 增量渲染、ResizeObserver 按需重绘（并正确释放）
实用性 10%	场景真实性与方案有效性	面向真实仓库；发现带行号与证据可直接定位；架构蓝图可作为评审材料
用户体验 10%	UI 设计 4%	设计令牌系统（颜色/间距/字体三档）、两套强调色、A4 一致的信息密度

评分维度（权重）	赛题考察点	本项目的实现与位置
	交互体验 3%	运行状态条、任务清单实时更新、空态文案按助手方言区分、430px 起自适应
	AI 交互体验 3%	SSE 流式渲染、思考与工具调用可视化、错误边界（失败不伪装）
社会价值与商业潜力 10%	社会价值 5%	降低代码阅读与架构分析门槛，可用于教学与开源维护；评估体系公开可复现
	商业潜力 5%	可插拔内核可扩展到测试生成、文档生成等更多助手；面向团队可私有部署

关于 RAG 的说明。赛题把 RAG 列为考察点。本项目没有采用向量数据库方案， 而是面向代码场景设计了另一套检索策略：**工具清单渐进披露 + 符号级读取 + 按需上下文。** 理由写在 4.4 与 4.8：代码检索的难点不是语义相似度，而是「准确的行号与结构」， 以及「不把整个仓库塞进上下文」。二者目标一致（让模型在有限上下文里拿到需要的知识）， 技术路径不同。

3 系统架构设计

3.1 总体架构

系统分为四层：浏览器端工作台、HTTP/SSE 服务层、Agent 运行时（Harness）、以及被 Agent 调用的工具与沙箱。 其中**工具层是 Agent 与外部世界（文件系统、命令、模型）之间的唯一通路** —— 所有读写都经过它，因此安全护栏只需在这一层收口。



图 3-1 总体架构：四层 + 唯一通路原则

3.2 Agent = Model + Harness

项目的核心抽象来自一个判断：**模型本身不可编程，能编程的是它外面的那层骨架。** 把这层骨架（Harness）设计好，同一个模型就能稳定地完成不同任务。 Harness 由五部分组成：

组成	职责	本项目的实现
上下文管理	决定模型每一轮看到什么：静态前缀 + 动态追加	系统提示与核心工具定义恒定；进度、状态、检索到的工具 schema 追加到轨迹末尾
工具接口	把能力以模型能理解的方式暴露	JSON Schema 参数；描述写「何时用 / 何时不用」；数组参数显式声明
约束	限制行为边界，防止失控	迭代上限、重复调用检测、连续被拒熔断、命令白名单、只读助手硬契约
验证	检查产出是否可信	引用核验（行号是否真读过）、参数校验、确定性规则交叉验证
纠正	出错后给出可操作的修复路径	错误信息四层（类型 + 参数 + 调用栈 + 建议）；失败显式而非伪装成空结果

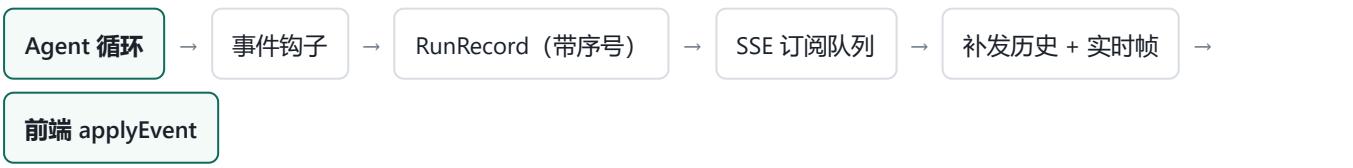
3.3 前端工作台

前端不做「一个助手一套页面」，而是**一套骨架 + 声明驱动的差异**：

- **骨架共享**：助手轨道（56px）+ 上下文栏 + 工作台 + 输入坞，四个助手都走这套结构
- **差异来自声明**：助手清单由后端 `GET /api/agents` 下发（未连后端时用内置兜底）， 前端按 `outcome` 选产出渲染器，按 `accent` 注入强调色变量
- **文案分两层**：与助手有关的（输入框提示、按钮、空态、运行状态）走「方言」表；与助手无关的（品牌、快捷键提示）保持不变
- **渲染器注册表 + 兜底**：不认识的产出形态按字段原样列出，不白屏、也不冒充实出

3.4 事件流：一条路径同时服务「实时」与「重放」

这是架构上刻意避免重复的一点：**打开一条还在跑的会话**与**回放一条已完成的会话** 走的是同一条代码路径 —— 服务端先补发历史事件，再接上实时推送。



每个事件带自增序号，因此断线重连可以按 `Last-Event-ID` 续传；终态后发送关闭帧并写 chunked 终止序列，客户端不会挂起等待。

3.5 关键架构决策

决策	选择	放弃的方案与代价
助手如何接入	声明式注册表：写下 id / 工具工厂 / 产出形态 / 强调色，调一次 `register`	放弃「写死在代码里的枚举」：那种做法每加一个助手都要改工具、渲染、路由、前端四处， 漏一处就是「跑得出来但显示成另一种产出」这种很难发现的问题
产出形态谁说了算	由助手自己声明 `outcome`，渲染层按注册表分派，未知形态走通用兜底	放弃「统一都叫发现」：架构运行若走审查渲染器，会打印「本次没有记录任何发现」—— 那不是事实，是把渲染器的不认识说成了产出为空，比不渲染更糟
产出如何存放	按类型分桶（`artifacts: dict[str, list]`），发现列表只是其中一个桶的视图	放弃单一 `findings` 字段：那会让第二种产出无处安放，且归档重放时丢桶
只读助手怎么保证	运行前硬检查：声明 `can_edit=False` 的助手若注册了中高风险工具，直接让运行失败	放弃「靠自觉不给写工具」：声明与能力脱钩时无人发现，等于没有边界
框架依赖	后端标准库实现 HTTP；前端原生 JS/CSS，无构建	放弃了框架带来的开发效率。代价是部分工作要自己写；收益是 **整套系统可被完整审计、零安装门槛、不受依赖漏洞影响** —— 对一个安全审计工具，这个交换值得

3.6 技术选型理由

层	选型	理由
模型	DeepSeek（OpenAI 兼容端点）	长上下文（约 100 万 token）支撑整仓库分析；缓存计价降低重复上下文成本；国内可直连
后端语言	Python 3.11+	代码分析生态与规则集实现成本低；标准库自带 HTTP/SSE/hashlib/cryptography 替代品
后端框架	无（标准库）	服务本身是被审计对象，依赖越少越好审；同源托管前端因此不需要 CORS
前端	原生 JS + CSS（ES2020+ / CSS 自定义属性）	零构建意味着评审者打开即用；无 npm 供应链风险；样式系统用令牌变量而非预处理器
实时通信	SSE	单向推送足够（Agent → 界面）；比 WebSocket 简单，且天然支持断线续传
持久化	JSON 文件（运行归档 / 账号表）	单机工具的数据量不需要数据库；JSON 可用任何工具检查，便于评审与调试
认证	scrypt + 内存会话	标准库可得的最强密码哈希之一；会话不落盘 = 不把长期凭据写到磁盘

4 AI 技术方案

本章是文档的重点：一个能跑起来的 AI 应用与一个可靠的 AI 应用之间的差距， 几乎全部落在这里 —— 上下文怎么组织、工具怎么设计、失控怎么兜住、产出怎么验证。

4.1 模型集成

4.1.1 集成方式

模型调用集中在单一的客户端模块中（`harness/model.py`），对上层暴露一个 「给消息与工具，返回一次决策」的接口。这样做的好处是：重试策略、退避、用量统计、 错误分类只写一遍，Agent 循环里不掺协议细节。

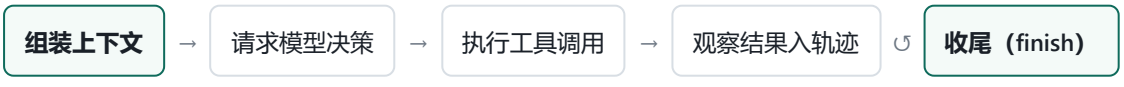
能力	实现要点
失败重试	只对可重试错误（限流、网络、5xx）重试；指数退避；重试过程以事件形式推给界面（用户能看到「在重试」而不是卡住）
用量统计	累计输入 / 输出 / 缓存命中 token，随运行结果一起归档 —— 成本可回溯
工具调用	走模型的 function calling 通道，参数以 JSON 对象返回；解析失败时（模型偶发输出非 JSON）归类为参数畸形错误并给出重试指引
降级	没有 API Key 时切换到启发式模型：仍走完整 Agent 循环与工具协议，产出由确定性规则给出 —— 界面与流程可以完整演示

4.1.2 选型理由

- **长上下文是硬需求**：架构分析需要跨文件理解调用链，小窗口模型会频繁触发压缩、丢失结构认知
- **缓存计价**：Agent 每轮都要重发系统提示与历史轨迹，缓存命中把成本与首字节延迟都压下来（本项目实测命中约 87%）
- **OpenAI 兼容协议**：换供应商只需改 base_url 与模型名，不绑定单一厂商

4.2 Agent 循环设计

循环的骨架是四步，每一轮都留下可观察的痕迹：



4.2.1 规划：把「打算怎么做」显式化

助手被要求用任务清单工具（`update_todo`）写下计划，并在推进中更新状态。 这不只是为了界面好看 —— 显式计划让「读了三个文件就下结论」这种偷懒行为变得可见， 也让用户能在中途判断方向对不对。

4.2.2 工具调用：能力按需可见

工具集分两档：**核心工具**进系统提示的静态前缀，随叫随到；**扩展工具**不进前缀， 需要用检索工具先搜出来再加载其 schema。这个设计解决了两个实际问题：

- **上下文占用**：把二十多个工具的完整定义全部前置，会占掉可观的上下文， 而在一次具体任务里多数工具根本用不到
- **选择质量**：候选工具越少，选错工具的概率越低

4.2.3 收敛：怎么收尾是设计出来的

收尾工具（`finish`）受两层约束：一是必须在总结里说明「检查了哪些范围、 为什么足以判定」，二是它**永不进熔断** —— 否则模型被护栏锁死时会连收尾都做不到， 结果一条产出都拿不回来。

> **踩过的坑（真实记录）。**早期版本的熔断用「累计被拒次数」判定， 一旦打开就再也不会关闭，模型被自己的护栏锁死 —— 连 `finish` 都被拒绝。 现在改成：判据是**连续**被拒次数（成功一次即归零），且收尾与产出类工具走前缀豁免。 这个 bug 的教训是：**护栏必须可恢复**，否则它就是故障源本身。

4.3 Prompt 工程

4.3.1 分层结构

层	内容	为什么放这里
身份与目标	你最擅长什么；本次任务的判据	最先进入上下文，决定整体基调
工具契约	核心工具定义（何时用 / 何时不用）	恒定不变，可被缓存
硬约束	只读边界、证据要求、不得编造行号	紧跟在目标之后，避免被具体任务细节冲淡
失败兜底	读不到文件怎么办、路径不存在怎么办	没有兜底规则时模型会即兴发挥，通常发挥得不好
风格与格式	产出字段要求、语言	放最后，不占早期注意力

4.3.2 工具描述的三条规矩

1. **写「何时用」也写「何时不用」**：只写能力会诱使模型把它当万能工具。 例如「只想看结构时用符号表工具，比读全文省 token」这类反向说明，能显著降低误用。
2. **数组参数必须明说**：光有 schema 的 `type: array` 不够 —— 实测模型连续多次把「只有一个条目的数组」写成裸字符串。描述里必须给出目标形态 （`['...']`，即使只有一项）。
3. **错误信息四层**：类型 + 参数 + 调用栈 + 修复建议。缺任何一层，模型要么猜，要么重复同样的错误调用。

> **一条被实测逼出来的纪律。**工具层**不做静默类型转换。** 把字符串自动包成数组看起来「贴心」，但它掩盖了模型的表达错误， 下次换个字段还会犯；正确做法是拒绝并在错误信息里给出目标形态。 这个原则同样适用于产出层：宁可让运行失败并说明原因，也不要生成一份看起来正常的错误数据。

4.4 工具接口设计

工具	作用	设计要点
`list_files`	列目录	目录不存在显式失败（不是「没有匹配」）；跳过点目录与依赖目录
`read_file`	读文件（带行号）	支持分段；行号前缀为证据引用服务；offset 越界显式报错
`grep`	正则搜索	非法正则给出转义提示；命中上限提示「结果不完整」而不静默截断
`read_symbols`	符号表（函数/类清单）	建立结构认知的高性价比手段，避免通读全文
`git_diff`	看变更	审查场景的起点：只看改动比通读全仓更有效率
`discover_tools`	检索扩展工具	渐进披露的入口；按需求检索而非全量列举
`record_*`	登记产出（发现/层/组件/决策/问题）	边确认边登记，不攒到最后；同名去重
`update_todo`	任务清单	让计划可见、可纠偏
`finish`	收尾	必须写范围与理由；统计口径随产出形态变化

4.5 上下文预算与压缩

长任务会超出上下文窗口，因此需要预算管理：

- **预算对象**统计静态前缀、历史轨迹与工具输出的 token 占用
- **压缩触发**：占用达到窗口的 80% 时，对较早的工具输出做摘要化处理，保留结论与证据，丢弃大段原文
- **截断提示**：单个工具输出过大时落盘并在上下文里只给摘要与路径，同时明确标注「已截断」，避免模型以为拿到了全部

4.6 缓存感知的上下文结构

这是本项目在成本与延迟上的关键优化，做法本身很简单，但需要纪律来维持：

恒定的静态前缀
系统提示 + 核心工具定义，顺序与内容**永不变化。** 任何动态内容（时间、进度、状态、 按需加载的工具 schema）都追加到轨迹末尾，绝不插进前缀。

为什么有效
主流模型服务按「前缀相同的最长部分」计费与加速。前缀稳定时， 每轮只有尾部新增部分需要重新计算 —— 本项目实测 KV 缓存命中约 87%， 同时降低了单轮延迟与成本。

维持这条纪律的难点。「把当前时间放进系统提示」是很自然的写法，但它会让前缀 每轮都变，缓存全部失效。因此项目里把这类信息统一交给「运行时状态栏」追加在轨迹尾部， 并以自检方式暴露前缀指纹，防止有人顺手破坏这个不变量。

4.7 AI 安全与可控性

模型会犯错，也会被诱导。以下四类护栏分别对应四种失控模式：

失控模式	护栏	恢复路径
陷入循环（反复调用同一个工具）	重复调用检测：连续 N 次相同调用即拒绝	拒绝信息明确告知「换个思路」；计数**连续**语义，换工具即归零
无限探索（读不完的仓库）	迭代上限（硬约束）	达到上限时如实标注「未收尾，结论可能不完整」，而不是假装完成
越权操作（不该改的代码被改）	只读助手硬契约 + 命令白名单 + 高风险工具需显式开关	声明与能力不符时运行直接失败，暴露问题而不是静默放行
编造证据（引用不存在的行号）	引用核验：结论中的 `file:line` 若不在本次读过的文件集合里，标为待复核	核验结果作为告警单列，不混进「发现」里冒充结论

4.7.1 敏感信息与越界

- **密钥文件护栏在工具层**：模型的读结果会进入上下文与报告，只挡浏览器是不够的
- **路径逃逸防护**：工作区解析统一收口，`..`、绝对路径、技能目录逃逸均拒绝
- **范围校验在服务端**：前端传什么都不能决定边界；越界项丢弃且不回显原文到模型上下文
- **只读助手的能力边界**：不注册任何中高风险工具，且由运行时检查兜底

4.8 规则与模型的交叉验证

本项目有一组确定性规则集（可检出硬编码密钥、危险函数调用、资源未释放等可判定的模式）。 它的作用有两个，第二个才是关键：

1. 不接模型时也能产出（离线模式的基础）
2. **作为模型结果的交叉验证器**：规则命中但模型没报 → 标记为疑似漏报； 两者都报 → 提高置信度。这是「能确定性判定的就不交给模型」这条原则的落地

> **为什么这是本项目的核心判断。** 语言模型的强项是语义理解与跨文件推理，弱项是精确的机械判定（某一行是否包含某个模式）。 把后者交给规则，前者交给模型，两边互相校验 —— 既避免了「用模型做正则的活」这种浪费，也让产出多了一道可验证的关口。

4.9 结构化产出

所有产出通过工具登记，参数即 schema。这样做的直接收益是：**模型不需要「写出一份格式正确的报告」， 它只需要填字段**；渲染、归档、评估都基于结构，而不是解析自由文本。

```
record_component(
  name         = "session",                  # 组件名
  layer        = "会话编排层",                # 所属层（必须先登记层）
  role         = "跨请求状态容器 + 请求编排",
  interfaces   = ["GET /users/{id}", ...],   # 数组：每项一个接口
  depends_on   = ["http-adapter", ...],      # 数组：每项一个已登记组件名
  failure_modes = ["连接池耗尽时排队直到超时"],  # 数组：每项一种失效后果
  rationale    = "为什么这样切"               # 必填的实质内容
)
```

产出层的两条质量约束：

- **依赖名对齐**：登记组件时把 `depends_on` 按「忽略大小写与首尾空格」 对齐到已登记组件的规范名，自依赖直接剔除，未登记的名字在回复里点名。 否则前端画连线时找不到目标，图上就会缺一条边 —— 这种「图缺了一块」极难事后发现。
- **未知形态兜底**：渲染层对不认识的产出按字段原样列出，不假装它是已知形态

5 核心功能实现

5.1 代码审查助手

5.1.1 工作流程

选定仓库与范围 → 看变更（git_diff） → 读文件 / 搜符号 → 登记发现 → 规则交叉验证 → **收尾报告**

5.1.2 发现的结构

字段	说明
严重级	阻断 / 警告 / 建议（三级；前端按级筛选）
位置	文件 + 行号 —— 必须指向缺陷所在行，可直接跳转
标题 / 说明	一句话结论 + 具体展开
证据	为什么判定为缺陷（引用代码、说明触发条件）

5.1.3 让产出可信的三个机制

- **证据必填**：没有证据的发现无法登记，从工具层杜绝「我觉得这里有问题」
- **引用核验**：登记后核验引用的行号是否真在本次读过的文件里，否则单列为待复核告警
- **规则交叉验证**：确定性规则命中的模式若模型未报，标记疑似漏报

5.2 架构设计助手

5.2.1 产出形态

架构助手的产出不是一段描述，而是四类结构化条目：

产出	内容
分层	每一层的名称、职责、边界提示
组件	所属层、角色、职责、对外接口、依赖、失效后果、切分理由
关键决策	选了什么 + 为什么 + **放弃了什么及代价**
开放问题	当前无法确定、需要人来定的问题（可标记是否阻塞）

> **为什么强调「放弃了什么」。**只写「我们选了 A」的文档无法评审 —— 评审要讨论的正是被放弃的那条路。因此决策记录强制包含备选方案与代价， 这让蓝图从「介绍现状」变成「可供讨论的方案」。

5.2.2 分层图的可读化

把结构画成图容易，画得可读不容易。实现中处理了这些细节：

- **层名走左侧栏位**：节点排在右侧。否则跨层连线必然横穿每层的说明文字
- **连线走廊实测**：跨层竖线走在「层名栏位与节点区之间的缝」，缝宽由 DOM 实测而来 （写死数值或不按栏位宽计算，窄屏下竖线会压进层名文字）
- **横段走空隙中线**：节点行之间留有间隔，横线取其中心，保证上下都有净空， 既不穿节点也不出层
- **同层连线的两种走法**：同行且中间不夹节点走「短 U 形」；否则走走廊直角折线 —— 固定下沉量的浅弧会从换行后的第二个节点身上穿过去
- **线型携带信息**：同层实线、跨层虚线（图例标明）

5.3 多助手隔离

「一个工作台 + 多个助手」最容易出的问题是串台。本项目在四个位置做了隔离：

位置	规则
正文归属	工作区正文归「当前选中的那一条会话」独占；切会话或新建任务先清空再渲染
历史过滤	助手运行列表与仓库计数在**服务端**按助手过滤，且两处用同一个口径 （否则会出现「写着 5 次却点不出来」）
未知助手	一律显式报错并附可用清单，绝不静默回退到审查助手 —— 「选错助手却跑出一份完整结果」比直接报错难发现得多
演示数据	每个助手一套演示数据；没有完整记录的会话如实说明，不假装有内容

5.4 账号与会话安全

环节	实现
密码存储	scrypt（n=2^{14} / r=8 / p=1） + 每账号独立随机盐，存 `scrypt$n$r$p$salt$hash`；比较用恒定时间函数；明文不落盘
登录防护	失败不区分「账号不存在」与「密码错」（防账号枚举）； 失败时也执行一次哈希运算以避免时间差泄漏；连续失败达阈值后短期锁定
会话	令牌只存服务端内存；Cookie 为 HttpOnly + SameSite=Strict； 退出时服务端吊销（不是仅清前端标记）
存储健壮性	账号文件原子写入（临时文件 + 替换），避免进程中断留下半截 JSON； 文件损坏时改名留档而不是静默清空
访问控制	默认不强制登录（本机工具直开即用）；开启后业务接口要求有效会话， 未登录返回 401 并带登录页地址，前端自动跳转

一个关于演示账号的工程细节。测试账号的密码在「前端常量」与「后端种子」两处出现， 任何一处漂移的后果都很具体：**用户照着页面上写的密码登录会失败。** 因此页面上的提示文本由前端常量渲染（不硬编码第三处），并加了一条测试把两处钉死 —— 改一处忘了另一处，测试立刻失败。

5.5 前端设计系统

方面	实现
设计令牌	颜色、间距、字体、层级全部走 CSS 自定义属性；改主题只改令牌
强调色分层	`--agent`（属于「这条运行 / 这个助手」）与 `--accent`（属于「这个产品」）分开；助手切换只改前者
响应式	三档断点（宽屏 / 中屏 / 窄屏 ≤900px 抽屉化）； 最小验证到 430px
可访问性	图标按钮有 `aria-label`；动态内容有 `role="status"`； 折叠操作有 `aria-expanded`；表单错误用 `aria-invalid` + 关联错误文本
空态与失败	空态文案按助手方言区分（「还没有审查记录」/「还没有设计记录」）； 失败显式提示且给恢复路径，不显示假数据

6 测试与部署

6.1 测试体系

175	0	~13s
回归测试项（全绿）	跳过的测试	全量测试耗时

测试文件	规模	覆盖内容
`test_harness.py`	主套件	主循环、上下文、工具、护栏、规则集、评估数据加载
`test_server.py`	服务层	真起 HTTP 服务的集成测试：路由、范围校验、归档、不泄漏密钥
`test_agents.py`	平台层	助手注册表、产出分桶、渲染分派、工具 schema 一致性、参数健壮性
`test_tools_common.py`	工具层	9 个公共工具的行为（路径逃逸 / 密钥拦截 / 分段 / 非法正则 / 目录不存在 / 白名单）+ SSE 集成
`test_auth.py`	认证层	哈希不落明文、账号枚举防护、限流、会话生命周期、HTTP 往返、强制登录模式

6.1.1 两条被测试守住的纪律

标注隔离
评估靶场的答案标注绝不能出现在被审计的代码里 —— 答案印在题目下面，指标就是假的。 有一条测试遍历所有靶场，检查代码里没有泄漏标注。

失败不伪装
「目录不存在」被说成「没有匹配内容」会驱动假阴性结论。 有一条测试断言这类情况必须显式失败并给出修复建议。

6.2 评估体系

AI 应用最难的部分之一是「怎么知道它做得好不好」。本项目用一套可复现的评估链路回答这个问题， 并有两条刻意坚持的纪律。

6.2.1 靶场与链路

8 个靶场仓库 → 批量运行 → 机器可读结果 → 人工复核 → **报告（脚本现算）**

8 个靶场覆盖不同难度：资源生命周期、并发安全、输入路径处理、边界条件、危险调用等， 每个靶场配有独立答案键（34 条标注）。

6.2.2 实测结果

口径	召回率	精确率	F1
自动匹配（同文件 + 行号 ±2）	86%	80%	82%
人工复核（逐条判定，带理由）	97%	95%	—
确定性规则基线（对照）	32%	—	—

> **为什么要给两个口径。**自动匹配用「同文件 + 行号容差」判定，会产生三类系统性误判： 同一缺陷行号略有不同（被算成一漏一多）、模型指出的真问题不在答案键里（被算成误报）、 相邻缺陷互相顶替。只报对自己有利的那个口径，等于把度量方式伪装成能力差异 —— 因此两个数字都要给，且人工复核的每条判定都写明理由。

6.2.3 度量本身的纪律

- **度量工具出 bug 时先怀疑度量。**本项目发生过两次：一次是匹配逻辑重复消费标注， 导致召回被低估约 10 个百分点；一次是分组口径把冗余算成误报，精确率从 100% 掉到 58%。 两次都不是模型的问题。
- **指标对行号容差高度敏感。**整体 F1 在容差 ±0 时为 61%，放宽到 ±4 时升到 95% —— 所以报单一容差下的数字没有意义。
- **单次跑分不可引用。**同一个靶场三轮的召回波动可达 100/100/50。 因此选型类结论必须跑 3–5 次看均值与波动。
- **报告不手抄。**交付文档中的数字由脚本从原始结果现算生成，避免手抄与数据脱节。

6.3 架构助手的三档难度实测

针对架构助手单独设计了难度刻度，用三个真实开源仓库验证能力边界 （psf/requests、pytest-dev/pluggy、celery/celery，规模 7k / 12k / 123k 行）：

档位	仓库	结论
中等	requests	正确识别出「requests 本身不做重试，重试在传输适配层的 urllib3」—— 这是一个「没读代码就一定会说错」的判定点，可作为能力校验的探针
中等	pluggy	8 条关键决策全部命中真实设计取舍（顺序决策放在注册时刻、用属性命名空间做宿主隔离、 刻意不做并发保护）
困难	celery	任务链路的九个环节齐全，并提出真实存在的结构问题（两套任务状态并存、 确认语义分裂在不同层、跨三层直接依赖）

实测暴露的两个反直觉结论（如实记录）。 ① **产出规模不随仓库复杂度缩放**：同一个仓库两次运行产出 9 个组件与 19 个组件（波动一倍），而简单仓库与复杂仓库的产出量级相同 —— 因此**不能把产出计数当作质量或难度的代理指标。** ② **困难案例显著更难收敛**：复杂仓库运行中出现「读了 43 个文件仍撞到迭代上限」，需要在产品上给出更明确的收尾引导。

6.4 部署与运行

方式	命令 / 说明
本地运行（推荐）	`cd backend && python -m harness.server --port 8000`，打开 `http://127.0.0.1:8000/`
命令行模式	`python -m harness -r <仓库> --goal "..."`；加 `--scan-only` 走纯规则扫描（不调模型）
离线模式	不配置 API Key 即可运行，走确定性规则 + 启发式模型
演示模式	直接打开 `index.html`，使用内置演示数据（无需后端）
运行依赖	Python 3.11+ 与 `httpx`；无 Node.js、无数据库
数据目录	`.runs/`（运行归档）、`.auth/`（账号表）—— 均不入库

6.5 代码质量与工程纪律

- **注释写「为什么」而不是「做什么」**：关键决策点都记录了放弃的方案与实测依据，并保留被验证为错的做法的说明（例如熔断为什么必须可恢复）
- **一次实现，多处使用**：公共工具只有一处实现（两个助手共享），重复实现必然导致行为分叉
- **安全边界用测试守住**：密钥不入库、标注不泄漏、失败不伪装，都有对应测试
- **提交历史保留真实开发过程**：可看到问题如何被发现与修复（含被推翻的结论）

7 创新点说明

7.1 技术创新点

创新点 1：助手即声明（Agent as a Declaration）

把「一个 AI 助手」抽象成一份声明：标识、系统提示、工具工厂、产出形态、强调色、只读边界。 接入新助手只需写这份声明并注册，**工具层、渲染层、HTTP 层、前端轨道都不需要改动。** 更进一步：声明是**可执行的契约**—— 声明只读的助手若注册了写工具，运行会在启动时直接失败， 而不是靠自觉。

创新点 2：产出形态由助手声明，渲染层注册表 + 兜底

产出不是「统一都叫发现」，而是由助手自己声明形态（发现列表 / 分层蓝图 / 其他）， 渲染层按注册表分派，遇到未注册的形态走通用兜底（按字段原样列出）。 这消灭了一类很难发现的问题：**把「渲染器不认识」说成「产出为空」。**

创新点 3：确定性规则与语言模型的交叉验证

「能确定性判定的就不交给模型」—— 规则集既独立产出，又作为模型结果的验证器： 规则命中而模型未报标记为疑似漏报，两者都报提高置信度。 实测中这一机制把规则难以覆盖的语义类缺陷 与 模型容易忽略的机械模式 各自补齐。

创新点 4：缓存感知的上下文结构

静态前缀（系统提示 + 核心工具定义）恒定不变，一切动态信息追加到轨迹末尾， 使模型侧 KV 缓存命中率实测达到约 87%，显著降低长任务成本与延迟。 项目还把「前缀指纹」做成自检项，防止后续改动无意破坏这个不变量。

创新点 5：可审计的零依赖架构

后端 HTTP/SSE 层用标准库实现（外部依赖仅 httpx），前端原生 JS/CSS 无构建。 对一个**安全审计类工具**而言这不是「简陋」，而是设计选择： 依赖越少越可审计、零安装门槛、不受供应链风险影响。整套系统可在无 Node.js、 无数据库的机器上完整运行。

创新点 6：面向代码场景的检索策略（替代向量 RAG）

代码检索的难点不是语义相似度，而是**准确的行号与结构。**项目采用三条路径替代向量检索： 工具清单渐进披露（先检索再加载 schema）、符号级读取（只看函数/类清单建立结构认知）、 按需上下文（读到的内容带行号，作为证据可核验）。目标与 RAG 一致（在有限上下文里拿到需要的知识）， 路径更贴合代码场景。

7.2 应用创新点

创新点 7：审查与设计共用一个可插拔内核

把两类差异很大的研发任务（找缺陷 / 讲结构）放进同一工作台，共享同一套 Agent 运行时、 工具协议、事件流与界面骨架，差异全部由声明表达。这验证了内核的通用性 —— 新增助手不再是「再造一个应用」，而是补一份声明。

创新点 8：架构蓝图的可评审化

架构助手的产出不是描述性文字，而是「分层 + 组件 + 决策（含放弃的方案与代价）+ 开放问题」。 其中**开放问题**尤其有价值：它明确标出「当前无法确定、需要人来定」的部分， 把 AI 的边界如实交付给使用者，而不是用流畅的语言掩盖不确定性。

创新点 9：面向评审的诚实性设计

一系列刻意的「不假装」： 失败必须显式（目录不存在 ≠ 没有匹配）、产出计数不当质量指标、 无法确定的问题单列、渲染器不认识就说认识不了、被中断的运行如实标注未完成、 演示数据与真实数据在界面上明确区分。**在 AI 应用里，可信度本身就是产品能力。**

创新点 10：评估驱动开发

项目自带完整的评估链路（靶场 → 批量运行 → 人工复核 → 报告现算），并在开发过程中 反复修正**度量本身**的错误（匹配逻辑重复消费标注、分组口径把冗余算成误报）， 最终形成「双口径报告 + 单次不可引用 + 数字不手抄」的纪律。

8 总结与展望

8.1 已交付能力

能力	状态
可插拔多助手内核	✅ 内置两个助手，接新助手 = 一份声明
代码审查助手	✅ 带证据的发现列表，8 靶场人工复核 R=97% / P=95%
架构设计助手	✅ 分层图 + 组件 + 决策 + 开放问题，三个真实仓库验证
实时交互	✅ SSE 流式事件，历史补发与实时推送同路径
安全与可控	✅ 密钥护栏、路径逃逸防护、范围服务端校验、引用核验、四类护栏
账号系统	✅ 注册 / 登录 / 退出，scrypt 哈希，可选强制登录
测试与评估	✅ 175 项回归 + 完整评估链路（含人工复核）
响应式界面	✅ 三档断点，430px 起可用

8.2 已知边界与不足

如实说明当前能力边界，这些是后续版本要解决的问题：

边界	影响与现状
未集成多模态	当前聚焦代码文本理解，未支持图像 / 语音 / 视频输入。 对于「看界面截图找问题」这类场景尚不支持
产出规模不稳定	实测同一仓库两次运行的组件数可相差一倍（9 vs 19）； 复杂仓库偶发「撞迭代上限未收尾」。已在文档中如实标注，不把产出计数当质量指标
归档未脱敏	运行归档会保存工具输出原文；若被审仓库在非密钥命名文件里夹带凭据， 会被明文归档（已记为待改进项）
会话不持久化	会话令牌只在服务端内存，服务重启需要重新登录 —— 这是「不把长期凭据写盘」的取舍
单机定位	服务默认只绑定回环地址，面向单机使用；未做多用户隔离与配额管理
评估样本量	8 靶场 34 条标注，够看难度梯度但不足以做模型选型； 答案键已知不完整（人工复核发现 6 处真缺陷未被收录）

8.3 后续计划

1. **收敛性改进**：为复杂仓库引入「先分层后细化」的两阶段策略，并在接近迭代上限时 主动提示收尾，降低「读了 43 个文件仍没结论」的概率
2. **产出规模校准**：把「组件粒度」显式写入助手声明（粒度指导）， 并对规模异常（过细 / 过粗）给出运行时提示
3. **多模态接入**：支持设计稿与界面截图作为输入，扩展「看图找实现问题」的场景
4. **归档脱敏**：对运行归档中的凭据模式做替换，进一步收紧数据卫生
5. **助手生态**：按同一份声明扩展测试生成、文档生成、依赖升级评估等助手， 验证内核的可扩展边界
6. **团队协作**：多用户隔离、运行配额与评审工作流（把当前的单机工具推进到团队可用）

> **一句话总结。** 本项目的价值不在于「用 AI 写了一个工具」，而在于把**如何让 AI 可靠地做研发工作** 这件事工程化：能力用声明表达、边界用运行时强制、产出用结构化收口、 错误用可恢复的护栏兜住、效果用双口径评估验证、不足如实写进文档。